

Exercise 3 — Build it with a durable working method

A self-paced companion for making the AI workflow itself durable

Stand-alone version of Exercise 3, for working on your own with any AI chat assistant. The classroom version runs entirely inside GitHub Copilot, with a skill and three agents pre-installed by an instructor script. This version teaches the same four ideas without any of that: three plain prompts, pasted into fresh conversations, in order.

What you are building

The exact same domain as Exercise 2, from the exact same two specifications — nothing about the software changes. What changes is the *process*: instead of one long conversation, the way you use the assistant becomes something you can inspect, approve, and reuse, not just something that happened once in a chat window.

The specifications

Same two files as Exercise 2, in the same place: `docs/spec/` inside an empty project folder.

- `sell-stocks-spec.md` — behaviour: preconditions, the FIFO rule, the money definitions, and acceptance criteria AC-01 ... AC-24.
- `domain-class-diagram.puml` — structure: classes, fields, methods, visibility, and relationships.

Why this exercise exists

Exercise 2 made the **behaviour** durable: it lives in a spec file instead of your memory of a conversation. But the **process** you used to turn that spec into code — the questions you asked first, the order you built things in, who checked the result and how — was still invisible. It happened once, inside one conversation, and vanished the moment the chat ended.

This exercise does for the *working method* what Exercise 2 did for the *domain*: it takes something that used to live only in a conversation and turns it into something you can read, review, and reuse — a **skill** for the repeatable method, and **role-scoped agents** for who is allowed to do what.

A concrete failure this catches

Picture Exercise 2 again. The specification never says what should happen at some edge case the acceptance criteria don't cover — say, a sale priced exactly at a lot's own purchase price. In one continuous conversation, the assistant can quietly pick an answer, write the code for it, then write a test that agrees with the code it just wrote — and you would very likely never notice, for the same reason Exercise 1's self-graded tests never failed: the actor that made the decision and the actor that checked it were the same conversation, with the same blind spot.

Exercise 3 puts two structural obstacles in that path:

1. **The planner has to write the assumption down, before any code exists.** If it silently invents behaviour, it shows up as a line in `plan.md` you are explicitly asked to approve — not as a fait accompli buried in a diff you skim past.
2. **The reviewer is a separate conversation that never saw the implementer's reasoning.** It checks the code against the specification and the diagram directly, not against “does this look like what I would have written” — because it never wrote anything.

Neither obstacle *guarantees* the assumption gets caught. What they guarantee is that it has to surface somewhere reviewable, instead of staying invisible inside one uninterrupted train of thought.

Exercise 2 vs. Exercise 3

DIMENSION	EXERCISE 2	EXERCISE 3
Domain inputs	Two specifications	The same two specifications
Process	One conversation, start to finish	Three role-scoped conversations, in order
Planning evidence	Whatever you remember from the chat	<code>plan.md</code> — written, dated, approved
Who checks the result	Often the same conversation that built it	An independent conversation that never wrote the code
Reuse next time	Retype the prompt	Paste the same Method block again, unchanged

Is it worth the overhead? Be honest about this

At this kata's size, Exercise 3 is genuinely more work than Exercise 2's thirty-line prompt — three prompts and a written approval instead of one. That overhead does not pay for itself on a task this small, and “Worth trying afterwards” at the end asks you to feel that directly.

It starts paying off at a different scale than this kata: when a change is large or risky enough that you need to *prove*, after the fact, who approved what before it was built — closer to how a bank already treats a pull-request approval or a change record than it might first appear. It also

pays off across time: a skill written once gets reused on the next feature; a prompt typed once gets forgotten.

Hold both of those in mind as you work through the rest of this document — the four ideas below are what carries that value, not the paperwork itself.

The four ideas — and why each one matters

1. **A skill is a versioned, repeatable method** — not a one-off prompt you retype each time. Write it once, review it like any other file in the repository, and reuse it unchanged on the next feature instead of reconstructing it from memory in a fresh conversation.
2. **Role separation is a permission boundary, not a suggestion.** A planner that cannot edit files cannot quietly start implementing before you've approved anything; a reviewer that cannot edit cannot “fix while reviewing” and blur checking into doing.
3. **A human checkpoint gates planning → implementation.** Nothing gets built until you've read the plan, changed at least one thing yourself, and written down that you approve it — turning approval into a real decision instead of a reflexive “looks good.”
4. **Independent review happens before any fix.** The reviewer reports findings first; you decide what goes back to the implementer — keeping “did we build the right thing” separate from “do I like what got built.”

Technical constraints

- Java 21, Maven, a standalone project with its own `pom.xml`, no parent.
- JUnit 5 only, in test scope. No Spring, no persistence, no REST layer.
- Package root: `com.neueda.portfolio.domain`.
- Every monetary amount as `BigDecimal`, scale 2, `HALF_UP` — never `double` or `float`.

The classroom version of this exercise targets Java specifically. If you'd rather work in another language, adapt this block the same way Exercises 1 and 2 do — the method below (skill, role separation, checkpoint) transfers regardless of language; only these bullets change.

The shared Method block

This is the “skill” for this exercise — a fixed, reusable description of *how* to work, independent of *what* gets built. In a tool with a skills feature, it lives in a file your assistant loads by itself. Here, you're the loader: paste this block, unchanged, at the top of all three role prompts below.

COPY THIS BLOCK

Method: Explore, Plan, Implement, Review.

Explore – read both specification files completely before proposing anything. Report missing inputs or conflicts between them instead of resolving them silently.

Plan – produce a complete proposal for a plan.md file (template supplied separately). Map every planned class to the class diagram. Map every acceptance criterion, AC-01 to AC-24, to a named test. Do not create or edit any implementation file in this phase.

Implement – follow an approved plan, in dependency order: build configuration, value objects, entities, exceptions, aggregate behaviour, then acceptance tests. Implement only what the diagram and specification actually describe. Keep the domain pure – no framework, no persistence, no REST layer. Run the tests as you go, then the full suite.

Review – read the specifications and the plan again, independently. Trace every acceptance criterion to executable evidence. Compare the implemented structure with the diagram. Check rejected operations for partial mutation, not only their return value. Report findings before proposing any fix.

The plan.md template

Create this file in your project now, before starting the Planner prompt. Its status stays **draft** until you personally change it to **approved**.

COPY THIS BLOCK

```
# Sell Stocks implementation plan

Status: draft

## Goal
[what must exist when the exercise is complete]

## Inputs reviewed
- [ ] sell-stocks-spec.md
- [ ] domain-class-diagram.puml

## Scope boundaries
In scope: [domain and test work required by the specifications]
Out of scope: [infrastructure and invented behaviour that must not be added]

## Specification questions or conflicts
[write "None found" only after reading both files completely]

## Ordered implementation tasks
- [ ] Task 1: ...
- [ ] Task 2: ...

## Acceptance-criterion coverage
One row per criterion, AC-01 through AC-24:
| Criterion | Planned test name | State/result evidence |
| --- | --- | --- |
| AC-01 | | |
...
```

```
## Verification plan
- [ ] Focused tests during implementation
- [ ] Full test run, exactly 36 tests
- [ ] Independent structure and behaviour review
- [ ] Full diff review

## Risks and assumptions
[without inventing decisions that belong to the specifications]

## Human checkpoint
- Decision: pending
- Approved by:
- Date/time:
- Conditions or requested changes:
```

Role prompt 1 — Planner

Start a **fresh conversation**. Paste the Method block, then this:

COPY THIS BLOCK

Using only the Explore and Plan phases of the method above, read the attached specifications and

return a complete proposal for plan.md, following the template exactly. Do not create or edit any

implementation file – return the proposed content only, for me to review.

[Attach or paste sell-stocks-spec.md and domain-class-diagram.puml here.]

The human checkpoint

Copy the returned proposal into your own `plan.md`. Then, before touching anything else:

1. change or refine **at least one item yourself** — a split task, an added risk, a corrected dependency order, a criterion the proposal under-specified;
2. confirm AC-01 through AC-24 all appear in the coverage table; and
3. fill in the Human checkpoint block: `Decision: approved`, your name, the date.

Do not continue until this file says `approved` in your own words, not the assistant's. This step is the point of the exercise — it is what makes approval a review act instead of a rubber stamp.

Role prompt 2 — Implementer

Start **another fresh conversation**. Paste the Method block, then this:

COPY THIS BLOCK

Using only the Implement phase of the method above, execute this approved plan exactly. If you

need to deviate from it, stop and tell me why instead of silently continuing.

[Paste your approved plan.md in full.]

[Paste the technical constraints block from this document.]

Run `mvn test` yourself once it reports done — don't take "the tests pass" on trust from the transcript.

Role prompt 3 — Reviewer

Start a **third fresh conversation** — not the implementer's. This matters: a reviewer that already wrote the code will tend to agree with itself, for the same reason Exercise 1's tests did.

COPY THIS BLOCK

Using only the Review phase of the method above, perform an independent review. Report findings before proposing any fix — do not edit or rewrite code in this conversation.

Check: whether every criterion from AC-01 to AC-24 has executable evidence; whether the exact numbers match the specification; whether rejected operations leave state completely untouched, not only their return value; whether the implemented classes, fields and methods match the diagram exactly, with nothing extra invented; and whether the project stays free of any framework or infrastructure code.

[Attach or paste both specifications, your approved plan.md, and the implementer's final code.]

Closing the loop

Read the reviewer's findings and classify each one: a verified defect, a suspected risk needing more evidence, an optional improvement outside the specification, or a false positive. Only findings **you** accept go back — and they go to the *implementer's* conversation, not the reviewer's.

Completion contract

#	REQUIREMENT
1	<code>mvn test</code> passes with exactly 36 tests, no failures or errors
2	AC-01 through AC-24 are traceable in test names or display names
3	The implementation matches the class diagram — nothing extra, nothing missing
4	<code>plan.md</code> was approved, in your own words, before any implementation started
5	The reviewer ran in an independent conversation and reported before any fix
6	Only findings you accepted were sent back for correction
7	<code>plan.md</code> still accurately describes what was actually built

A green test run alone does not satisfy this contract — items 4 through 6 are about the *process*, and nothing in the code proves they happened.

Worth trying afterwards

- **Run the reviewer in the implementer's own conversation** instead of a fresh one, and watch it agree with itself. It's the same blind spot as Exercise 1's self-written tests, wearing a different costume — an evaluator that already committed to an answer rarely reverses itself.
- **Approve a plan with a task deliberately left out**, then run the implementer. Does it notice the gap and ask, or build past it silently?
- **Count the overhead.** Exercise 2 was one prompt of about thirty lines. This exercise was three role prompts plus a written, human-approved checkpoint. Was that worth it at this kata's size? At what size would it be?

What's next

The domain became durable in Exercise 2. The working method became durable here. Exercise 4 keeps both and changes what they get pointed at: the same unchanged domain, now wrapped in a REST API and a real database.